

ThinStation-NG 7.2 — Compilación de una imagen universal en Fedora

Generación de una imagen **ISO + PXE** (híbrida **UEFI + Legacy BIOS**) de cliente ligero con sesión **RDP en modo kiosco** (diálogo de IP al arrancar), acceso **SSH** y máxima compatibilidad de hardware.

- **Repositorio:** github.com/thinstation/thinstation-ng · rama **7.2-Stable**
- **Verificado contra:** rama oficial **7.2-Stable** (mayo 2026)
- **Documento origen:** [20260528-thinstation-ng-tutorial-fedora - con icono personalizado.pdf](#)

1. Visión general

ThinStation-NG es un sistema operativo de cliente ligero de código abierto. El objetivo de este proceso es generar **una sola ISO** que arranque en el mayor número posible de equipos, sin afinarla por modelo, con una sesión RDP que pide la IP del servidor al arrancar. Todo el trabajo se hace por línea de comandos en un equipo Fedora; el entorno gráfico vive **dentro de la imagen**, no en la máquina que compila.

El proceso tiene **dos fases**:

Fase	Herramienta	Duración	¿Repite al recompilar?
Poblar el chroot (Fedora 42 + herramientas TS)	<code>./setup-chroot</code>	~25–40 min	No — queda en disco
Compilar la imagen (paquetes + kernel + squashfs + ISO)	<code>./build</code>	~20–40 min	Sí — usa caché

Ubicación de los artefactos: tras compilar quedan en `<repo>/ts/build/boot-images/grub/`. GRUB es el único método de arranque no obsoleto en la 7.2.

2. Entorno requerido: Fedora

La compilación **solo funciona en Fedora**, por dos motivos verificados en el propio código:

- **El chroot se construye con `dnf`**. El script usa `dnf install --installroot ... --releasever=42` para descargar la base del sistema desde los repos de Fedora. Otras distros no tienen `dnf` ni esos repos → esta fase falla.
- **El empaquetado final necesita root real**. El último paso, que crea la imagen de arranque EFI, usa dispositivos `loop` y `mount`, que requieren privilegios reales sobre el kernel.

Si NO usas Fedora: instala una VM con Fedora (edición Server, sin escritorio) y haz todo el proceso dentro de ella. Compilar fuera de Fedora hará que el proceso falle. En adelante «el equipo Fedora» se refiere indistintamente a una instalación física o a esa VM. Todo se ejecuta como **root**.

Requisitos previos

- **8 GB de RAM** (recomendado; con 4 GB irá lento).
- **40–50 GB de disco libre** (el README pide 30, pero una imagen universal ocupa más durante el build).
- **Privilegios de root** (`sudo -i`).
- **Conexión a internet** (descarga la base de Fedora 42 y los paquetes).

3. Guía paso a paso

Paso 1 — Instalar dependencias

`coreutils` (que aporta `chroot`) ya viene en Fedora. El resto (`mksquashfs`, `xorriso`, `grub2`) lo arrastra `setup-chroot`.

```
sudo -i                                # trabajar como root
dnf install -y git util-linux which dosfstools
```

Paso 2 — Clonar el repositorio

`7.2-Stable` es una **rama**, no una etiqueta. El directorio resultante será la raíz del chroot; colócalo en una partición con espacio suficiente.

```
cd /root
git clone --branch 7.2-Stable \
  https://github.com/thinstation/thinstation-ng.git
cd thinstation-ng
```

Working dir: `/root/thinstation-ng`

Paso 3 — Poblar el chroot

`./setup-chroot` descarga la base de Fedora 42, instala los paquetes del sistema, monta `/dev`, `/proc` y `/sys` dentro del chroot y abre su shell. Tarda **~25-40 min** la primera vez.

```
./setup-chroot
```

Al terminar verás el prompt del chroot: `[root@TS_chroot]/#`. Es normal ver algunos avisos cosméticos (paginadores, `systemd/dconf` dentro de un chroot); no impiden la compilación.

Paso 4 — Ir al directorio de build

Dentro del chroot, `/build` es un enlace a `/ts/build`, donde están `build.conf`, `thinstation.conf.buildtime` y `./build`.

```
cd /build
```

Paso 5 — Configurar `build.conf` (paquetes)

Activa (descomenta) estas líneas. Nombres verificados contra `7.2-Stable`. Para máxima compatibilidad se incluyen los **tres drivers de vídeo abiertos** (Intel, AMD, Nvidia) además de los de VM, y `sshd`:

Línea a activar	Función
<code>package xorg7</code>	Servidor X con driver universal <i>modesetting</i> (KMS)
<code>package xorg7-intel</code>	Driver de GPU Intel
<code>package xorg7-amdgpu</code>	Driver de GPU AMD/ATI
<code>package xorg7-nouveau</code>	Driver de GPU Nvidia (abierto)
<code>package xorg7-qxl</code> / <code>xorg7-vmware</code>	Drivers de GPU para máquinas virtuales
<code>package autonet</code>	Red automática (DHCP)
<code>package freerdp</code>	Cliente RDP + diálogo de IP + reconexión
<code>package locale-es_ES</code>	Teclado y locale en español
<code>package sshd</code>	Servidor SSH (dropbear) para acceso remoto al cliente
<code>package ssh</code>	Cliente SSH (para conectar desde el thin client a otros equipos)
<code>package git</code> · <code>package nano</code>	Utilidades en consola: control de versiones y editor ligero
<code>package alsa</code> · <code>package pipewire</code> · <code>package xfce4-pulseaudio-plugin</code>	Sonido (pila de audio + icono de volumen) — necesarios para redirigir audio/micrófono por RDP

Nombres que NO existen en 7.2-Stable (causan error): `xorg` → usar `xorg7` · `netbase` → usar `autonet` · `xorg-video-vesa` → no existe (KMS va en `xorg7`) · `dialog` / `xterm` → no son paquetes TS-NG.

Firmware: mantén `param allfirmware false` en `build.conf`. El firmware completo (~402 MB) se inyecta en `lib.squash` mediante el parche `fastboot-mangle` (ver **\$4b**), **no** en el initrd. Poner `allfirmware true` re-infla el initrd y **rompe el arranque BIOS**.

Paso 6 — Configurar `thinstation.conf.buildtime`

La clave: el paquete `freerdp` incluye `/etc/cmd/freerdp.getip`, que dispara el diálogo de IP cuando no hay servidor fijo. **Sin scripts propios**. Sustituye la línea por defecto `SESSION_0_TYPE=xfwm4` y deja:

```
# Audio
AUDIO_LEVEL=90
MIC_LEVEL=0

# Sesión RDP en modo kiosco
SESSION_0_TYPE=freerdp
SESSION_0_AUTOSTART=on
# NO poner SESSION_0_FREERDP_SERVER -> dispara el dialogo de IP
SESSION_0_FREERDP_OPTIONS="/multimon /dynamic-resolution +clipboard /network:auto
/microphone:sys:pulse /audio-mode:0"
FREERDP_CERTIGNORE=on      # equivale a /cert:ignore
RECONNECT_PROMPT=On       # al cerrar, vuelve al dialogo de IP

# Red / zona horaria / idioma
NET_USE=BOTH
NET_USE_DHCP=on
TIME_ZONE=Europe/Madrid
LOCALE=es_ES
```

Teclado: `KEYMAP=es` **NO existe** en 7.2. El teclado español se activa con `package locale-es_ES + LOCALE=es_ES`.

Audio y micrófono en la sesión RDP

Las opciones de audio en `SESSION_0_FREERDP_OPTIONS` (con la pila `alsa` + `pipewire` + `xfce4-pulseaudio-plugin` del `build.conf`) controlan la redirección de sonido por RDP:

- `/microphone:sys:pulse` — redirige el **micrófono local** del thin client al servidor (para videollamadas, dictado, etc.). Antes se usaba `/sound:sys:pulse` (audio de salida); se cambió a `/microphone` para capturar la entrada de micrófono.
- `/audio-mode:0` — el audio de la sesión se reproduce en el **cliente** (modo 0 = *redirect to local*), que es lo normal para un puesto físico.
- `AUDIO_LEVEL=90` / `MIC_LEVEL=0` — nivel inicial de volumen y de micrófono al arrancar.

Multimonitor (puestos físicos de doble monitor)

Para que un puesto físico con **dos monitores extienda** el escritorio (no que lo clone), el mecanismo válido en 7.2 es `USE_XRANDR` + `XRANDR_OPTIONS`, **no** las variables `SET_RESOLUTION_MULTIMONITOR_*`:

```
# Multimonitor: extiende si hay 2 monitores, normal si hay 1
USE_XRANDR=On
XRANDR_OPTIONS="dualscreen"
```

Comportamiento (verificado en `packages/base/etc/thinstation.functions`, función `use_xrandr`):

- **2 monitores conectados** (puesto físico) → extiende con `xrandr --left-of` (primer monitor primario). Al **no** fijar `SCREEN_RESOLUTION`, cada monitor usa su **resolución nativa** — ideal para parque heterogéneo. La opción `/multimon` de la sesión RDP reparte ambos.
- **1 monitor** (VM o puesto simple) → resolución normal. El valor `dualscreen` es inerte aquí.

Variables muertas: `SET_RESOLUTION_MULTIMONITOR_EXPAND` y `SET_RESOLUTION_MULTIMONITOR_AUTOSCALE` aparecen en las plantillas de `conf/*/thinstation.conf.buildtime` pero **ningún script las lee** en 7.2 (`grep -rn MULTIMONITOR_EXPAND /build/` solo las encuentra en las plantillas). Poner `mirror`, `right`, etc. **no hace nada** — usa `USE_XRANDR` / `XRANDR_OPTIONS`.

Paso 7 — Compilar la imagen universal

No uses `--allmodules`: inflaría el initrd y rompería el arranque BIOS (ver §4b). La cobertura universal de módulos + firmware la aporta el parche `fastboot-mangle` metiéndolos en `lib.squash`. Las dos flags evitan confirmaciones (aceptan licencias y descargan binarios sin preguntar):

```
# En /build, dentro del chroot · ~20-40 min (!SIN --allmodules!)
./build --license ACCEPT --autodl 2>&1 | tee /build/build.log
```

Si se interrumpe, **relanza el mismo comando** (usa caché). Al terminar, verifica que la ISO existe y no quedó en 0 bytes (debe mostrar varios cientos de MB):

```
ls -lh /build/boot-images/grub/thinstation-efi.iso
```

Para sacar la ISO a otra máquina o a un USB, cópiala desde: `/root/thinstation-ng/ts/build/boot-images/grub/thinstation-efi.iso`. Si compilas en una VM, puedes traerla con `scp`.

Paso 8 — Arrancar el cliente (UEFI obligatorio · RAM · USB por escritura directa)

Dos requisitos **no negociables** verificados en la puesta en marcha real:

La ISO es híbrida BIOS+UEFI

La imagen arranca **tanto en UEFI como en Legacy BIOS** (El Torito BIOS + UEFI + MBR isohíbrido; ver §4b para cómo se logró). En físico, **desactiva Secure Boot** (el GRUB de ThinStation-NG no va firmado → con Secure Boot activo se rechaza en silencio); puedes arrancar por la entrada **UEFI:** o por la Legacy del medio. Para saber en qué modo arrancó un sistema ya en marcha: `bash [ -d /sys/firmware/efi ] && echo "Arranque UEFI" || echo "Arranque BIOS/Legacy"`

Histórico: hasta junio 2026 la imagen fallaba en BIOS (se quedaba sin gráfico) porque el initrd pesaba 433 MB y GRUB-BIOS corrompía su mitad alta al leerlo del CD. Resuelto adelgazando el initrd (§4b). Si reaparece el síntoma "BIOS sin gráfico" en una build propia, sospecha un initrd grande: compruébalo con `xz -dc /build/boot-images/initrd/initrd | cpio -t | wc -l` y revisa que no se haya colado `--allmodules` ni `allfirmware true`.

RAM suficiente (la imagen arranca en RAM)

Con `param fastboot lotsofmem` el squashfs (~300 MB) se vuelca a RAM para arrancar rápido. Si la VM o el equipo tiene **poca memoria**, el sistema de ficheros en RAM se llena y la sesión gráfica **no puede escribir su config** → cae a consola con errores `No space left on device` (al copiar `/etc/skel`, `write lastlog failed`, etc.). La NIC y el kernel funcionan, pero **no hay entorno gráfico**.

- **Mínimo sano: 4 GB de RAM** (initrd ~45 MB + `lib.squash` ~900 MB que se vuelca a RAM con `lotsofmem` + capa de escritura tmpfs). Subir la RAM de la VM y rearrancar lo resuelve sin recompilar.
- **Para equipos con poca RAM**, alternativa: en `build.conf` cambiar `param fastboot lotsofmem` por `param fastboot true` (no vuelca el squashfs a RAM; arranca algo más lento pero consume mucha menos memoria) y recompilar.

Grabar el USB por escritura directa (NO Ventoy)

Graba la ISO directamente en el pendrive en modo imagen, no como fichero copiado dentro de otro gestor. ThinStation-NG localiza su squashfs (~300 MB, que contiene `lightdm` y toda la pila gráfica) recorriendo los dispositivos como si fueran un CD (`boot_device=cd0` en el cmdline). Para que ese autodescubrimiento funcione, el USB tiene que llevar el ISO9660 **escrito tal cual**, igual que un CD-ROM real:

- **Windows — Rufus (verificado):** en *Elección de arranque* selecciona `thinstation-efi.iso`, **Esquema de partición = GPT**, **Sistema de destino = UEFI (no CSM)** y pulsa **EMPEZAR**. Si Rufus pregunta tras pulsar EMPEZAR, elige «Escribir en modo Imagen DD» (no «modo ISO»). Verificado con Rufus 4.14 → arranca el entorno gráfico en el MSI físico.
- **Windows — balenaEtcher:** siempre escribe en modo imagen; alternativa simple si Rufus diera guerra.
- **Linux — dd:** `bash sudo dd if=thinstation-efi.iso of=/dev/sdX bs=4M status=progress oflag=direct conv=fsync` (`/dev/sdX` = el USB **completo**, sin número de partición; ⚠ borra el USB).

Después arranca por la entrada **UEFI:** del USB (CSM y Secure Boot desactivados, ver arriba).

⚠ **No uses Ventoy con esta ISO.** Ventoy no expone la `.iso` como un dispositivo real: la mapea con su driver virtual, y ThinStation-NG **no encuentra su squashfs** → arranca solo con el initramfs (kernel + red + consola mínima) pero **sin lightdm ni entorno gráfico**. Síntoma exacto: queda en login de consola; `command -v lightdm` vacío; `df -h` sin ningún squashfs/overlay montado (solo tmpfs). Se reprodujo en **normal mode** y **memdisk mode** (`grub2 mode` da «No bootfile found for UEFI!»). La escritura directa de arriba lo resuelve. Verificado en MSI físico, junio 2026.

La contrapartida es que el USB queda dedicado a esta única ISO (Rufus DD / `dd` borran el multiarranque). Para esta imagen compensa: es lo único que arranca de forma fiable.

4. Por qué la imagen es universal

Una sola ISO para todo el parque, sin afinar por modelo — pero **todo el peso (módulos + firmware) viaja en `lib.squash`, no en el `initrd`** (ver §4b), para que arranque también en BIOS:

Elemento	Qué aporta	Cómo se activa
Todos los módulos del kernel en <code>lib.squash</code>	Drivers de red, almacenamiento y vídeo (DRM/KMS) de casi cualquier hardware	parche <code>fastboot-mangle</code> (inyecta <code>/lib/modules/\$KV</code>)
Todo el firmware en <code>lib.squash</code>	Wi-Fi, Bluetooth y firmware de GPU (incl. GSP de Nvidia)	parche <code>fastboot-mangle</code> (inyecta <code>/lib/firmware</code>)
Driver X universal + específicos	<code>modesetting</code> (en <code>xorg7</code>) funciona con cualquier GPU con KMS; Intel/AMD/Nvidia complementan	paquetes <code>xorg7</code> , <code>xorg7-intel/amdgpu/nouveau</code>

La GPU del equipo (`i915` / `amdgpu` / `nouveau`) y su firmware están en `lib.squash`, que se despliega a RAM al arrancar; el driver carga **después** del montaje y `modesetting` inicializa la pantalla. Así se cubre el hardware físico **sin** meter 400+ MB en el `initrd` (lo que rompía el arranque BIOS).

Contrapartida — RAM: `lib.squash` pesa ~900 MB y con `lotsofmem` se vuelca a RAM. Mínimo sano ≥ 4 GB; en equipos muy justos, `param fastboot true` (no vuelca a RAM) a cambio de algo más lento.

Los perfiles `machine qemu/VMWare/Virtualbox` de `build.conf` aportan los módulos de esas VMs **al `initrd`** (ya que no se usa `--allmodules`); déjalos.

4b. Arranque híbrido BIOS+UEFI y reparto `initrd` / `lib.squash`

La ISO arranca **igual en UEFI y en Legacy BIOS** (una sola imagen). Que funcione dependió de entender el reparto entre el `initrd` y `lib.squash`:

- **`initrd` (~45 MB):** lo mínimo para llegar al CD y montar `lib.squash` (`isofs`, `squashfs`, `loop`, `overlay` + `busybox` + el `init` de `fastboot`). GRUB-BIOS lo carga **entero** en RAM sin corromperlo.
- **`lib.squash` (~900 MB):** TODO lo demás — los **4807 módulos** del kernel + el **firmware completo** (~402 MB) + Xorg + librerías + apps. Con `param fastboot lotsofmem` se vuelca a RAM al arrancar; los drivers/firmware de GPU/red se cargan **de forma perezosa** después → cobertura total sin inflar el `initrd`.

El fallo que había (`initrd` gigante en BIOS)

Por defecto ThinStation mete **módulos y firmware dentro del `initrd`** (la lista `fastboot/lib-boot` los marca como "quédate en el `initrd`"). Con `--allmodules` + `allfirmware true` el `initrd` llegaba a **433 MB**, y **GRUB en modo BIOS corrompe la mitad alta de un `initrd` tan grande al leerlo del CD** (UEFI usa otro cargador y no se entera). El kernel abortaba con `"Initramfs unpacking failed: XZ-compressed data is corrupt"` → no montaba `lib.squash` → sin gráfico.

Diagnóstico clave: el `RAMDISK` de `dmesg` era **idéntico** en BIOS y UEFI (432,7 MB — se cargaba entero), pero solo BIOS fallaba al desempaquetar, y el `xz -t` en userspace validaba el fichero entero → **corrupción de lectura específica de BIOS con `initrd` grande**, no del fichero ni del compresor.

Los tres cambios que lo arreglan

1. **Enrutado en `grub.cfg`** (plantilla `boot-images/templates/grub/default/grub.cfg`): en BIOS local fuerza **siempre** `boot_device=cd0`, para que la initramfs localice `lib.squash` por la **etiqueta de volumen** (`/dev/disk/by-label/$CDVOLNAME`, rama `iso()` de fastboot), de forma **agnóstica al nombre de dispositivo**. La versión anterior solo reescribía a `cd0` cuando `$root` empezaba por `hd`; en el **MSI físico** (BIOS + USB) GRUB resolvía `$root=fd0`, que no casaba con `^hd` ni con ningún caso del dispatch de fastboot (`cd*` / `hd96` / `hd31` / `hd*` / `tftp` / `http`) → caía en `boot_device UNKNOWN!` → nunca montaba `lib.squash` (arranque a consola, sin módulos ni red). Forzar `cd0` en **todo** arranque BIOS lo resuelve (en UEFI se deja `$root`, que ya funcionaba). Copia de referencia en `ThinStation/grub.cfg`. Caso completo en `s8b`.
2. **initrd single-block** (`ts/build/build` línea 506): `xz --threads=1 --check=crc32` (antes `--threads=0`, multihilo → multi-bloque). Higiene (el descompresor XZ del kernel espera single-block); no era la causa de fondo, pero es lo correcto.
3. **initrd mínimo + cobertura en `lib.squash`** (la clave): se compila **sin** `--allmodules` (el initrd conserva solo el set mínimo de módulos) y un parche en `fastboot/fastboot-mangle` inyecta el árbol completo `/lib/modules/$KV` + todo `/lib/firmware` en `lib.squash` en lugar del initrd. `build.conf` lleva `allfirmware false`. **Parche reproducible:** `ThinStation/parche-fastboot-mangle-paso2.sh` — aplícalo en el chroot tras cada `setup-chroot` si se regenera (es idempotente).

Verificar que la ISO es híbrida: `bash xorriso -indev thinstation-efi.iso -report_el_torito plain`

-> "El Torito boot img : 1 BIOS ..." y "El Torito boot img : 2 UEFI ..."

Y en la VM, por SSH, tras arrancar (UEFI o BIOS): `bash find /lib/modules -name '*.ko' | wc -l # ~4807` (todo el set, desde lib.squash) `ls /lib/firmware/nvidia # firmware de GPU presente (incl. GSP)`

5. Acceso y depuración por SSH

El paquete `sshd` (dropbear) del Paso 5 permite administrar el cliente y sacar logs. La imagen es de solo lectura, así que SSH se incluye **al compilar**, no se instala sobre el cliente arrancado. El paquete trae `systemctl enable dropbear` (arranca solo) y su `ExecStart` no lleva `-w`, por lo que **permite login de root**.

Credenciales por defecto (definidas en `build.conf`): usuario `root` / contraseña `root` (`param rootpasswd`); usuario normal `tsuser` / contraseña `tsuser` (`param tsuserpasswd`). **Cámbialas** en `build.conf` antes de cualquier despliegue real.

```
ssh root@IP_DEL_THINCLIENT
```

```
# Logs utiles (se pueden copiar con scp):
```

```
dmesg | grep -iE 'drm|i915|amdgpu|radeon|nouveau'
```

```
lspci | grep -iE 'vga|display|3d'
```

```
ls -la /dev/dri/
```

```
journalctl -b | grep -iE 'xorg|lightdm|drm|EE'
```

```
cat /var/log/boot.log ; cat /var/log/messages.log
```

```
# driver KMS cargado
```

```
# GPU del equipo
```

```
# ¿card0? si no, el kernel no ligó KMS
```

```
# arranque de X y display manager
```

```
# arranque y syslog
```


El log de Xorg NO está en `/var/log/Xorg.0.log`. El FS es de solo lectura, así que Xorg escribe bajo `/run/user/<uid>/` (logs por usuario en `/run/user/<uid>/applications/`, ficheros `session.*` y `xorg.*`). Para localizarlo sin saber el uid: `bash find /run /tmp /home -iname 'xorg*.log' -o -iname 'session*' 2>/dev/null` Si X ni siquiera arrancó, lo más fiable es `journalctl -b | grep -i xorg` y revisar `/dev/dri/`.

Seguridad: la clave de host de dropbear viene fijada en el repositorio (igual para todos). Para depurar es indiferente, pero en producción conviene regenerarla y no dejar login de root con la contraseña por defecto, sobre todo si el equipo queda expuesto a la red.

6. Recompilación posterior

El chroot persiste en el directorio del repositorio. Para recompilar tras un cambio:

```
cd /root/thinstation-ng
./setup-chroot      # detecta que ya esta instalado y entra directo
# Si setup-chroot regeneró /build/fastboot/fastboot-mangle, re-aplica el parche $4b:
#  bash <ruta>/parche-fastboot-mangle-paso2.sh
cd /build && ./build --license ACCEPT --autodl
```

Si solo cambiaste `thinstation.conf.buildtime` (sin tocar `build.conf`), el build solo regenera las imágenes de arranque — es lo más rápido.

7. Mecanismo del diálogo de IP nativo

El «pedir IP al arrancar y reconectar al cerrar» no requiere scripts propios; está en el paquete `freerdp`:

Elemento	Qué hace
<code>freerdp.getip</code>	Su presencia activa <code>CMD_GETIP=true</code> (incluido por defecto)
<code>read_options()</code>	Con <code>CMD_GETIP</code> y <code>\$SERVER</code> vacío, pone <code>ALLOW_SERVER_EDITS=true</code>
<code>dialog_get_server_address()</code>	Muestra un diálogo GTK editable para teclear la IP
<code>xfreerdp</code>	Se lanza con la IP y las opciones de <code>SESSION_0_FREERDP_OPTIONS</code>
<code>check_reconnect()</code>	Al cerrar la sesión, con <code>RECONNECT_PROMPT=0n</code> vuelve al diálogo

8. Resolución de problemas

Síntoma	Causa / Solución
Cae a consola con <code>No space left on device</code> (al copiar <code>/etc/skel</code> , <code>write lastlog failed</code>)	El FS en RAM se llenó: poca RAM . La imagen arranca en RAM (<code>fastboot lotsofmem</code>). Sube la VM/equipo a ≥4 GB (sin recompilar) o usa <code>param fastboot true</code> y recompila. Ver Paso 8
En físico no arranca / se queda sin gráfico	La ISO es híbrida (UEFI y BIOS). Si falla, desactiva Secure Boot . "BIOS sin gráfico" en una build propia con <code>--allmodules</code> o <code>allfirmware true</code> = initrd demasiado grande (§4b): compila sin <code>--allmodules</code> y con <code>allfirmware false</code>
Queda en login de consola sin gráfico; <code>command -v lightdm</code> vacío y <code>df -h</code> sin <code>squashfs/overlay</code> (solo <code>tmpfs</code>)	El USB se grabó con Ventoy : no expone la ISO como dispositivo real y ThinStation no monta su <code>squashfs</code> → arranca solo con el <code>initramfs</code> . Graba la ISO directa (Rufus modo Imagen DD / balenaEtcher / <code>dd</code>), sin Ventoy. Ver Paso 8
Igual que arriba (<code>df</code> sin <code>squashfs</code>) pero el USB SÍ está grabado en DD y <code>cat /proc/cmdline</code> muestra <code>boot_device=fd0</code> (o cualquier valor que no sea <code>cd*</code> / <code>hd*</code>)	GRUB mal-detecta el dispositivo en BIOS desde USB y <code>fastboot</code> no reconoce el <code>boot_device</code> (<code>UNKNOWN!</code>). Fix en <code>grub.cfg</code> : forzar <code>boot_device=cd0</code> en todo arranque BIOS (no solo si <code>\$root</code> casa <code>^hd</code>). Recompilar. Ver §4b y §8b
Equipo se queda en login de consola sin diálogo, pero el squashfs SÍ está montado (<code>df -h</code> lo muestra)	Primero descarta RAM (fila del <code>No space</code>). Si hay RAM de sobra: Xorg no levantó — el log no está en <code>/var/log/Xorg.0.log</code> sino en <code>/run/user/<uid>/</code> (ver §5). Revisa <code>journalctl -b \ grep -i xorg</code> (no screens found), <code>dmesg \ grep -i drm</code> y <code>ls /dev/dri/</code> (¿ligó un driver KMS?, ¿hay <code>card0</code> / <code>card1</code> ?)
ISO no se genera / 0 bytes; xorriso: 'Boot image file is empty'	El paso EFI no pudo usar <code>loop/mount</code> : no estás en un entorno con root real. Compila en Fedora o VM Fedora, no en entornos restringidos
<code>setup-chroot</code> falla al descargar la base	Casi siempre no estás en Fedora (no hay <code>dnf</code> ni repos compatibles)
Error: package X does not exist	Nombres incorrectos en <code>build.conf</code> . Usa <code>xorg7</code> y <code>autonet</code> ; <code>dialog</code> / <code>xterm</code> no son paquetes TS-NG
El build se para pidiendo confirmación	Usa <code>./build --license ACCEPT --autodl</code> (sin <code>--allmodules</code>)
No aparece el diálogo de IP (con X funcionando)	Probablemente quedó <code>SESSION_0_FREERDP_SERVER</code> en buildtime, o el tipo de sesión no es <code>freerdp</code>
No entro por SSH al cliente	Confirma <code>package sshd</code> , que el cliente tiene IP (<code>ip a</code>) y usa <code>root</code> / <code>root</code> (o tu <code>rootpasswd</code>)

8b. Caso real resuelto — equipo MSI (junio 2026)

La puesta en marcha en el **MSI físico** tuvo **dos obstáculos** distintos con el **mismo síntoma superficial** (arranca a consola, `df -h` sin `squashfs/overlay`, sin módulos ni red). Conviene conocerlos por separado porque la solución es diferente.

Obstáculo 1 — USB grabado con Ventoy

Síntoma: ISO que **funciona en VMware** pero en el MSI se queda en login de consola sin gráfico.

Causa: se arrancaba desde un **USB con Ventoy**, que no expone la `.iso` como dispositivo real (la mapea con su driver virtual). ThinStation-NG no encontraba su `squashfs` y arrancaba **solo con el initramfs**. Cambiar de `memdisk` a `normal mode` en Ventoy no bastó: el problema es Ventoy en sí.

Confirmación (consola `root` del equipo): `command -v lightdm` vacío; `df -h` solo `tmpfs`; `free -m` con RAM de sobra (descarta RAM); `dmesg | grep -i drm` con `nouveau` ligando KMS y `/dev/dri/card1` presente (descarta GPU/driver).

Solución: grabar la ISO **directamente** en el USB en **modo imagen/DD** (Rufus «Imagen DD», balenaEtcher o `dd`), sin Ventoy. Ver Paso 8.

Obstáculo 2 — `boot_device=fd0` en BIOS desde USB (el que faltaba)

Ya con el **USB grabado en DD**, el MSI **seguía** cayendo a consola en **arranque BIOS** (en UEFI sí arrancaba). Mismo `df -h` sin squashfs.

Diagnóstico (tty2 del propio MSI, sin red → fotos de consola): - `find /lib/modules -name '*.ko' | wc -l` → 0 módulos; `/dev/dri/` inexistente. - `df -h / mount` → **ningún squashfs ni overlay** (solo `tmpfs`) → `lib.squash` no se montó. - `free -m` → 16 GB, 15,5 GB libres → **no** es RAM. - `dmesg` → `xhci_hcd`, `usb-storage` y `uas` cargados; USB enumerado → **los drivers del initrd están bien**, no faltan módulos. - `blkid` → el medio es `/dev/sdc`, `LABEL="ThinStation"`, `TYPE="iso9660"` → la ISO está, accesible. - **La pista clave:** `cat /proc/cmdline` → `boot_device=fd0` (¡una disquetera fantasma!).

Causa raíz: GRUB, al arrancar el ISO híbrido **desde USB en Legacy BIOS**, resolvió su dispositivo raíz a `fd0` en vez de `hd0`. El `grub.cfg` solo reescribía a `cd0` cuando `$root` casaba `^hd`, así que `fd0` pasaba intacto al kernel. Y en el dispatch de `iso()` de fastboot (`/etc/init.d/fastboot`, líneas ~204-214) `fd0` no es `cd*`, ni `hd96` / `hd31`, ni `hd*`, ni `tftp` / `http` → `boot_device UNKNOWN!` → nunca entra en la rama que monta `/dev/disk/by-label/$CDVOLNAME` → `lib.squash` no se monta → initramfs pelado. (En la VM no pasa: el medio es un `cd0` óptico virtual real.)

Solución (la del título de §4b, cambio nº1): en `grub.cfg` (plantilla `boot-images/templates/grub/default/grub.cfg`) forzar **siempre** `boot_device=cd0` en BIOS, sin el `regexp "^hd"`. La rama `iso()` localiza el medio por su etiqueta `LABEL=ThinStation`, agnóstica al nombre de dispositivo, y monta el USB real. UEFI se deja con `$root` (ya funcionaba).

```
set bootdev="$root"
if [ "$grub_platform" = "pc" ]; then
-   if regexp "^hd" "$root"; then
-       set bootdev="cd0"
-   fi
+   set bootdev="cd0"
fi
```

Recompilar (solo cambió la plantilla de arranque → camino rápido): `cd /build && ./build --license ACCEPT --autodl`.

Resultado: ✅ arranca y muestra el escritorio RDP **en VM y en el MSI físico, tanto por UEFI como por BIOS** (verificado 2026-06-05). Comprobación tras arrancar: `cat /proc/cmdline` muestra `boot_device=cd0`, `find /lib/modules -name '*.ko' | wc -l` ≈ 4807 y `df -h` ya lista el `squashfs`.

9. Referencia rápida de comandos

Dónde	Comando	Propósito
Fedora (root)	<code>dnf install -y git util-linux which dosfstools</code>	Dependencias
Fedora (root)	<code>git clone --branch 7.2-Stable https://github.com/thinstation/thinstation-ng.git</code>	Clonar repo
Fedora (root)	<code>cd thinstation-ng && ./setup-chroot</code>	Poblar / entrar al chroot
Chroot	<code>cd /build && ./build --license ACCEPT --autodl 2>&1 \\\ntee /build/build.log</code>	Compilar imagen (sin <code>--allmodules</code> ; ver §4b)
Chroot	<code>ls -lh /build/boot-images/grub/thinstation-efi.iso</code>	Verificar la ISO
Cliente	<code>ssh root@IP_THINCLIENT · cat /var/log/Xorg.0.log · dmesg \\\ngrep -i drm</code>	Acceso y diagnóstico

Anexo A — Personalizar el logo de arranque (`watermark.png`)

El logo centrado en la parte inferior mientras carga ThinStation es la marca de agua (*watermark*) del tema **spinner** de Plymouth. ThinStation-NG instala `plymouth-theme-spinner` y sustituye únicamente su `watermark.png`, así que personalizar el logo se reduce a **reemplazar ese único fichero** en el repositorio y recompilar. El resto del tema (el indicador de progreso) se mantiene.

Dónde está el fichero

Es el mismo fichero visto desde tres sitios. Editas el del repositorio; el de la imagen arrancada es de solo lectura.

Contexto	Ruta
Host Fedora	<code>/root/thinstation-ng/ts/build/packages/plymouth/build/extra/lib64/plymouth/themes/spinner/watermark.png</code>
Dentro del chroot	<code>/build/packages/plymouth/build/extra/lib64/plymouth/themes/spinner/watermark.png</code>
Imagen arrancada (solo lectura)	<code>/usr/lib64/plymouth/themes/spinner/watermark.png</code>

No edites el logo en el thin client arrancado: el sistema de ficheros es de solo lectura. Reemplaza el PNG en el repositorio (host o chroot) y vuelve a compilar.

Conserva tamaño y formato

El `watermark.png` original es un PNG de **250 × 54 px** con transparencia (**RGBA**). Tres condiciones al sustituirlo:

- Mantén el nombre exacto `watermark.png`.
- Conserva el fondo transparente (PNG en modo RGBA).
- Conserva las dimensiones ($\approx 250 \times 54$ px). El tema spinner dibuja la marca **a su tamaño real sin escalarla**: si es mayor invade/recorta la pantalla, si es menor se ve diminuta. Encaja tu logo dentro de un lienzo del mismo tamaño (centrado, con relleno transparente) en lugar de cambiar el tamaño del lienzo.

Adaptar cualquier logo a 250×54 conservando proporción y transparencia con ImageMagick (`dnf install ImageMagick` si no lo tienes):

```
# copia de seguridad del logo original
cp watermark.png watermark.png.orig

# encajar TU logo en un lienzo de 250x54, centrado y transparente
magick mi-logo.png -resize 250x54 -background none \
    -gravity center -extent 250x54 watermark.png

# comprobar: debe decir 250 x 54 y RGBA
identify watermark.png
```

En versiones antiguas de ImageMagick usa `convert` en lugar de `magick`.

Aplicar el cambio (recompilar)

Como has modificado un fichero de un paquete (no solo `thinstation.conf.buildtime`), hace falta un build normal; usa la caché, así que será más rápido que la primera vez:

```
cd /root/thinstation-ng
./setup-chroot      # entra al chroot ya existente
cd /build && ./build --license ACCEPT --autodl
```

Nota: no existe una opción documentada en `build.conf` para cambiar el tema de Plymouth; reemplazar `watermark.png` es la vía soportada para personalizar el logo de arranque.

Documento generado a partir del tutorial verificado contra el repositorio oficial `thinstation/thinstation-ng` rama `7.2-Stable` (mayo 2026), incorporando las lecciones de la puesta en marcha real: host Fedora, imagen universal de máximo hardware, arranque híbrido BIOS+UEFI verificado en VM y en equipo MSI físico (2026-06-05), acceso SSH y logo personalizado.

NOTAS

Arranque BIOS — RESUELTO Y VERIFICADO EN FÍSICO (2026-06-05)

- La ISO arranca en **UEFI y Legacy BIOS** (ver §4b). Validado en **VM** (BIOS+UEFI) **y en el MSI físico** (BIOS+UEFI), con RDP gráfico, 4807 módulos + firmware completo presentes. El último obstáculo (`boot_device=fd0` en BIOS desde USB) se cerró forzando `cd0` en `grub.cfg` (§4b/§8b).

A vigilar

- Equipo que "superó la RAM" (8 GB, Gigabyte H510M S2H V2): con `lib.squash` ~900 MB volcándose a RAM (`lotsofmem`), vigilar memoria; si cae por RAM, probar `param fastboot true`. ¿Fallo de placa?
- UEFI en físico:** el fix de §4b solo toca la rama BIOS; UEFI mantiene `boot_device=$root` (funciona en VM y en el MSI). Si algún equipo fallara en UEFI con el mismo síntoma (`df` sin squashfs), habría que extender el enrutado `cd0` también a UEFI.